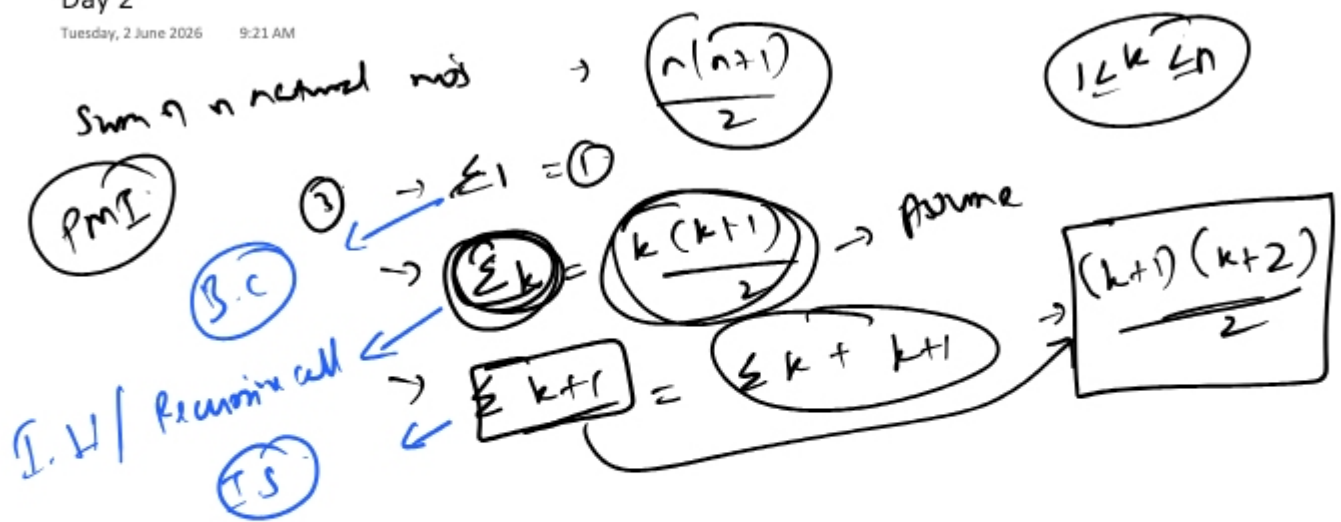




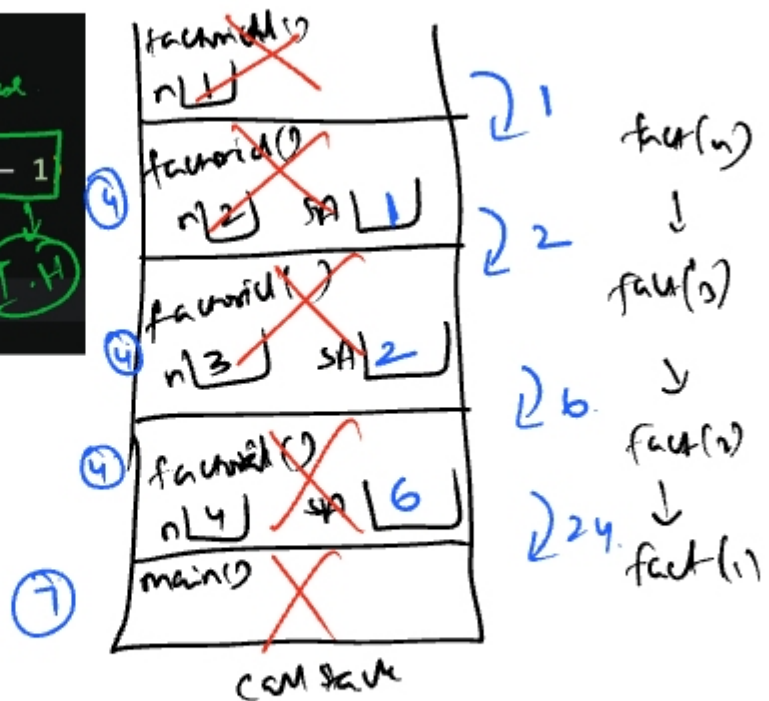
```

1 def factorial(n):
2     if n == 1: → Base case
3         return n
4     smallAns = factorial(n-1)
5     return n * smallAns
6 I.S.
7 print(factorial(4)) I.H.

```

Console:

24



User → name
 → acno → Auto incrementing
 → balance → 1000
 → pin.

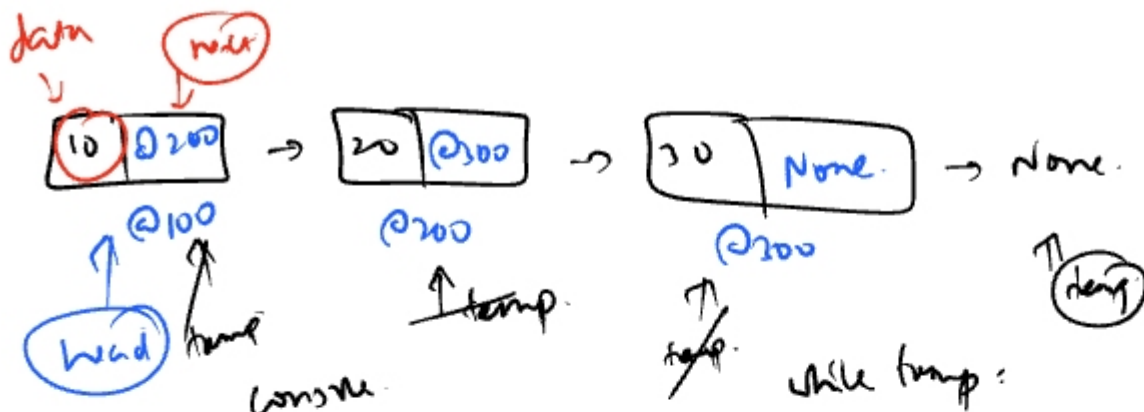
→ credit(amount).

→ debit(amount)
 ↳ Ask pin (3 times).

u = User("ax", 2579)

↳ Minimum balance 1000

↳ print User()



Console

@100	10	@200
@200	20	@300
@300	30	None

while temp:

print(temp, temp.data,
 temp.next)

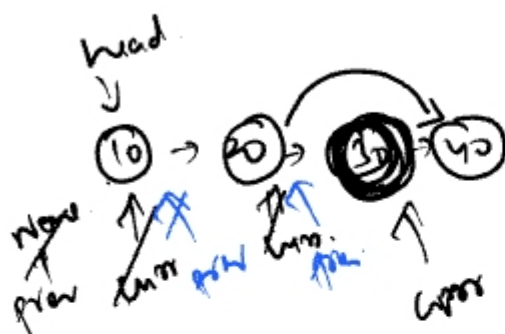
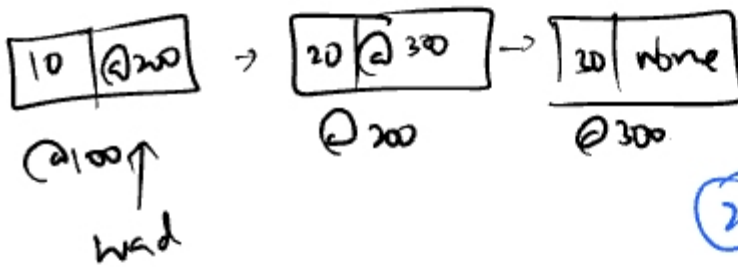
temp = temp.next



tail.next = node.

```

21 def printList(head) :
22     if head is None :
23         return
24     print(head.data , end = ">")
25     printList(head.next)
26
27 head = getInput()
28 printList(head)
  
```



val 30

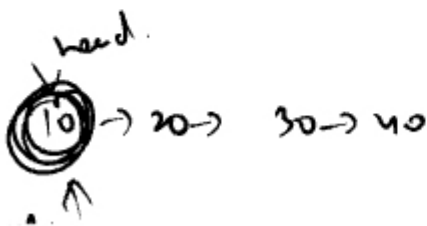
removeNode(head, val):

while curr:

if curr.data == val:

prev = curr

curr = curr.next



val 10

None
↑
prev
↓
curr

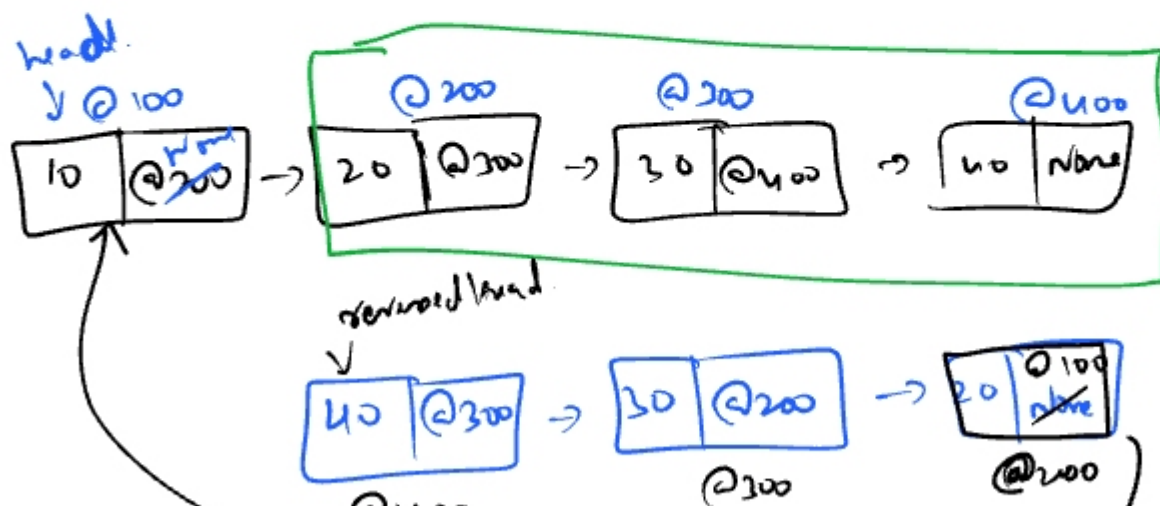
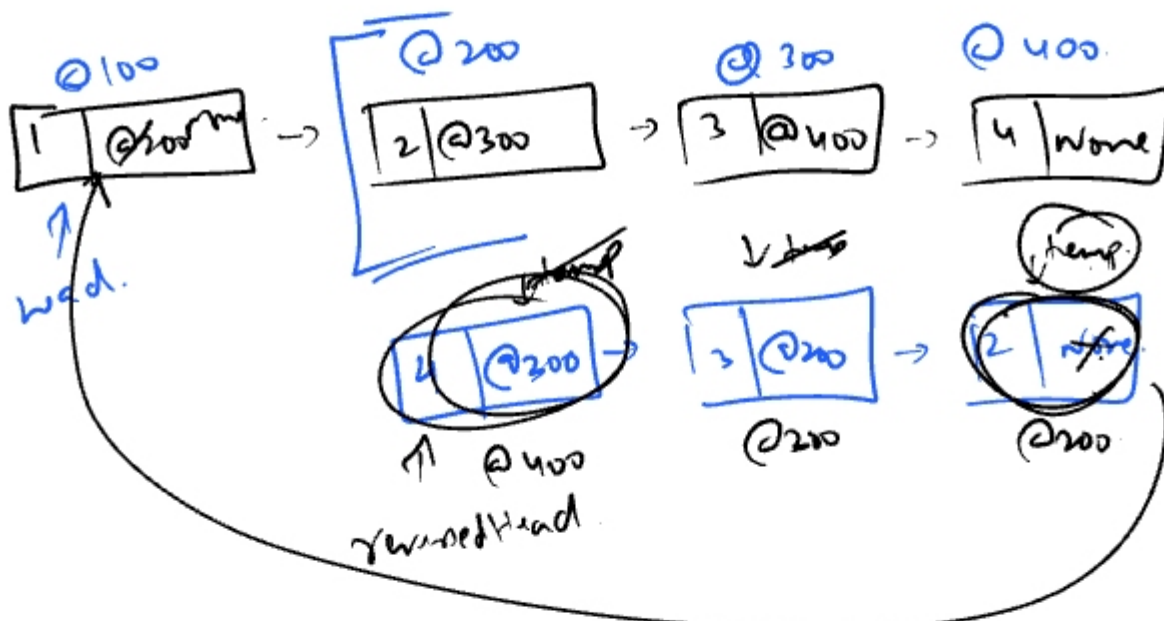
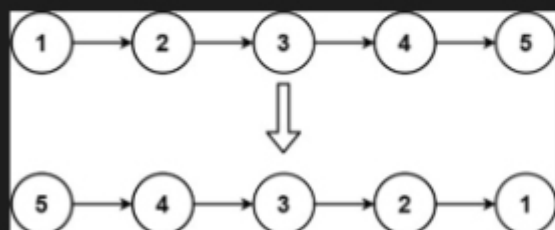
206. Reverse Linked List

Solved

Easy Topics Companies

Given the `head` of a singly linked list, reverse the list, and return the reversed list.

Example 1:



6400

```
6 class Solution:
7     def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
8         if head is None or head.next is None :
9             return head
10        reversedHead = self.reverseList(head.next)
11        head.next.next = head
12        head.next = None
13        return reversedHead
```

↓
p.No: 206

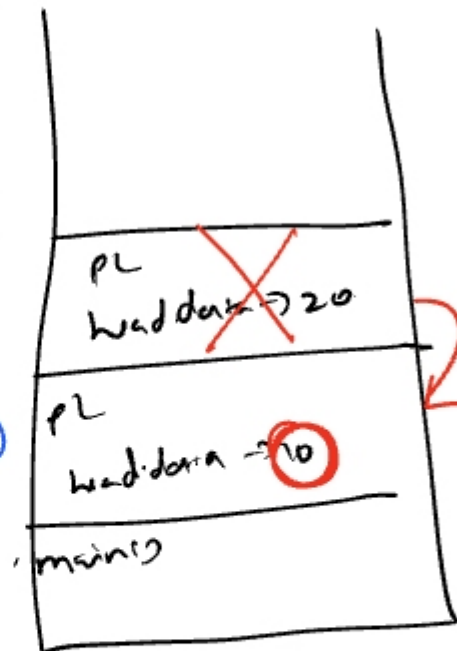
↓
T.C $\rightarrow O(n)$

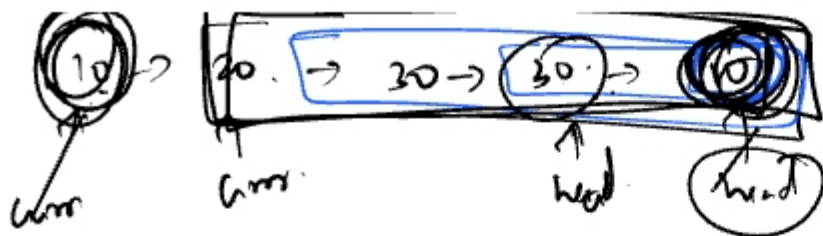
↓
S.C $\rightarrow O(n)$
↓
all stack space

```
21 def printList(head) :
22     if head is None :
23         return
24     print(head.data, end = "->")
25     printList(head.next)
26
27 head = getInput()
28 printList(head)
```

10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40

25



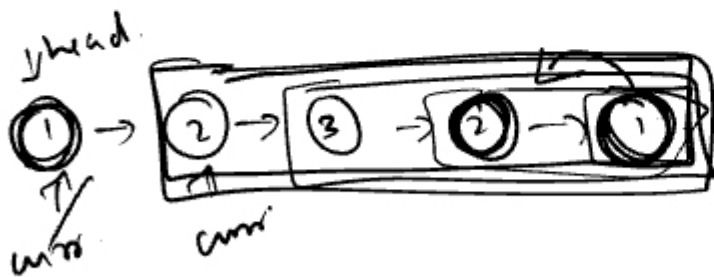


B-C → if list is empty

True

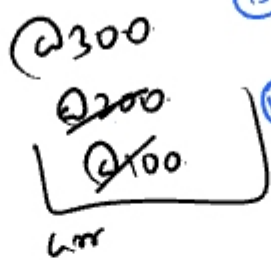
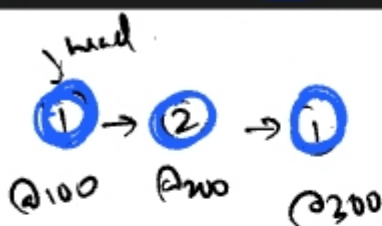
→ call recursion (head.next)

True



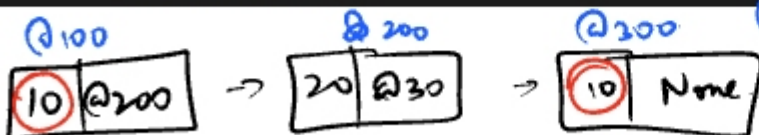
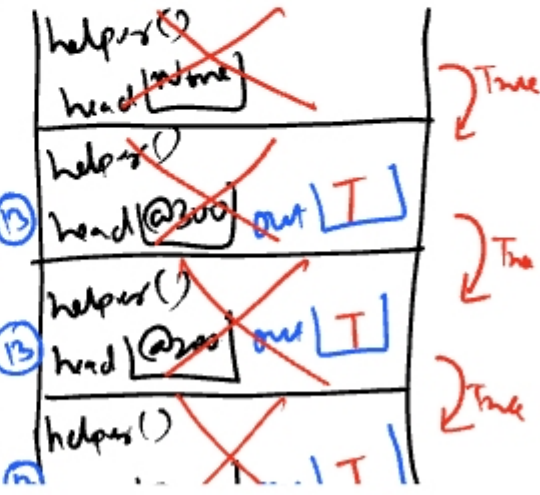
```

6 class Solution:
7     def isPalindrome(self, head: Optional[ListNode]) -> bool:
8         current = head
9         def helper(head):
10             nonlocal current
11             if head is None:
12                 return True
13             output = helper(head.next) and current.val == head.val
14             current = current.next
15             return output
16         return helper(head)
    
```



```

6 class Solution:
7     def isPalindrome(self, head: Optional[ListNode]) -> bool:
8         current = head
9         def helper(head):
10             nonlocal current
11             if head is None:
12                 return True
13             output = helper(head.next) and current.val == head.val
14             current = current.next
15             return output
16         return helper(head)
    
```





143. Reorder List

Medium

Topics

Companies

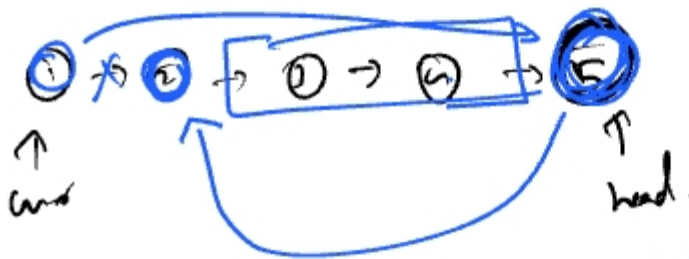
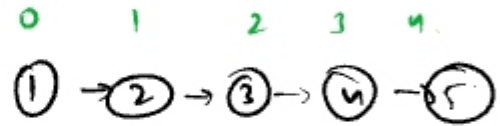
You are given the head of a singly linked-list. The list can be represented as:

$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$

Reorder the list to be on the following form:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

You may not modify the values in the list's nodes. Only nodes themselves may be changed.



nextNode = curr.next

curr.next = head

head.next = nextNode

nextNode = curr.next

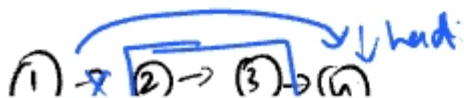
curr.next = head

head.next = nextNode

update current

current = nextNode

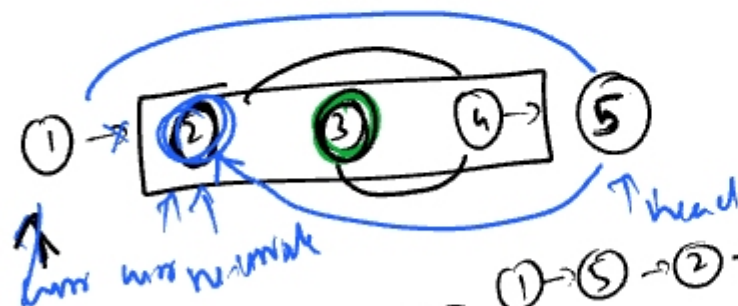
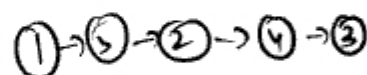
nextNode = curr.next



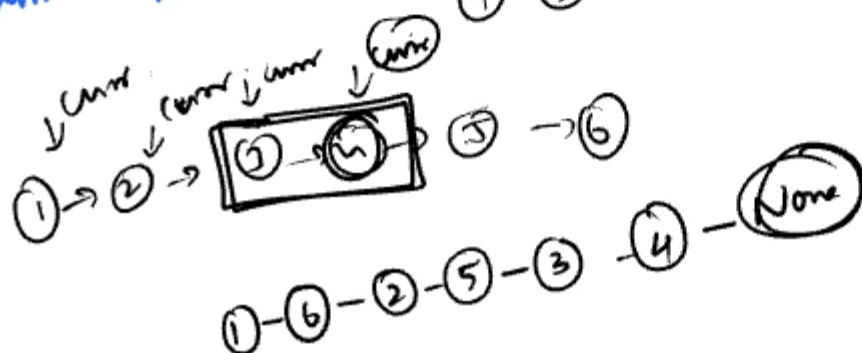


current → head.

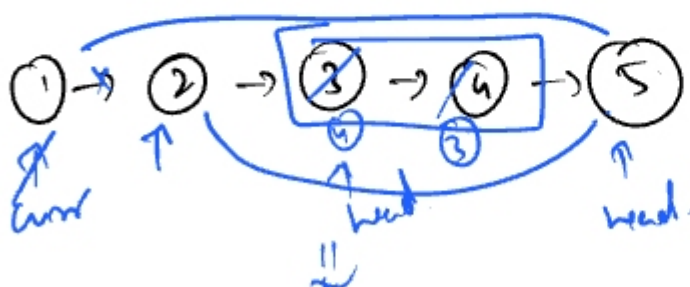
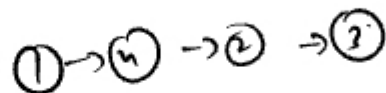
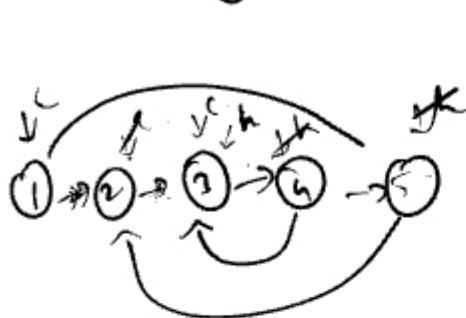
curr.next = head.
head.next = nextNode
curr = nextNode.



nextNode = curr.next
curr.next = head.



head.next = nextNode
current = nextNode.



```
def reorderList(self, head: Optional[ListNode]) -> None:
    """
    Do not return anything, modify head in-place instead.
    """
    current = head
    def helper(head):
        nonlocal current
        if head is None:
            return
        helper(head.next)
        if current is None:
            return
```




```

if current == head or current.next == head :
    current = None
    head.next = None
    return
nextNode = current.next
current.next = head
head.next = nextNode
current = nextNode

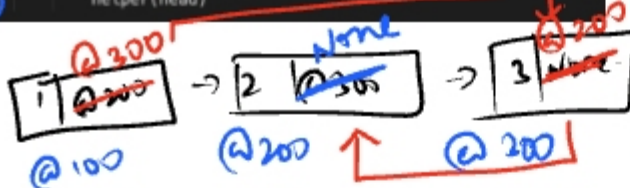
helper(head)

```

```

6 class Solution:
7     def reorderList(self, head: Optional[ListNode]) -> None:
8         current = head
9         def helper(head):
10             nonlocal current
11             if head is None:
12                 return
13             helper(head.next)
14             if current is None:
15                 return
16             if current == head or current.next == head:
17                 current = None
18                 head.next = None
19                 return
20             nextNode = current.next
21             current.next = head
22             head.next = nextNode
23             current = nextNode
24
25         helper(head)

```



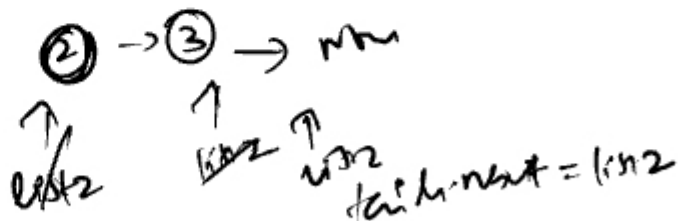
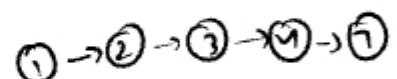
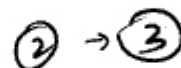
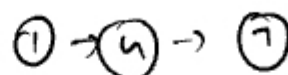
21. Merge Two Sorted Lists

Easy Topics Companies

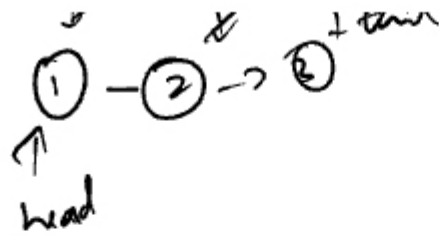
You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.



list1 ...



None
↑
12

```
def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
    if list1 is None:
        return list2
    if list2 is None:
        return list1
    head = None
    tail = None
    if list1.val < list2.val:
        head = list1
        tail = list1
        list1 = list1.next
    else:
        head = list2
        tail = list2
        list2 = list2.next
    while list1 and list2:
        if list1.val < list2.val:
            tail.next = list1
            tail = tail.next
            list1 = list1.next
        else:
            tail.next = list2
            tail = tail.next
            list2 = list2.next
    if list1:
        tail.next = list1
    else:
        tail.next = list2
    return head
```

→ Finding the head

→ P.No: 21

→ T.C → $O(m+n)$

→ S.C → $O(1)$

70. Climbing Stairs

Solved ✓

Easy Topics Companies Hint

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

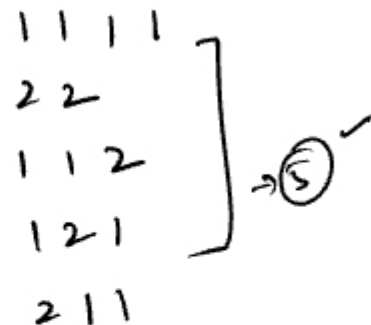
Input: $n = 2$

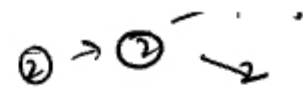
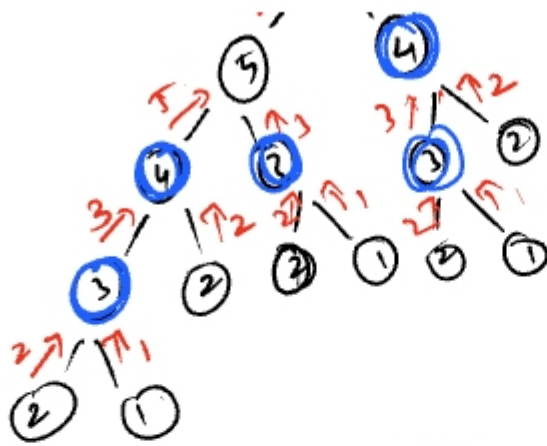
Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step
2. 2 steps

④

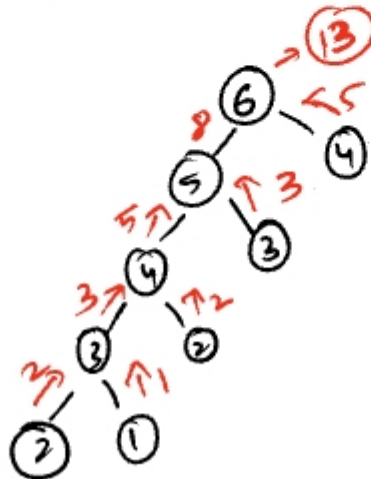




Problem

↓
Repeated subproblems

↓
DP → Memorization



0	1	2	3	n	5
			3	5	8

① → ①

② → ②

```
class Solution:
    def climbStairs(self, n: int) -> int:
        def helper(n, dp):
            if n == 1 or n == 2:
                return n
            ans1 = -1
            if dp[n - 1] == -1:
                ans1 = helper(n - 1, dp)
                dp[n - 1] = ans1
            else:
                ans1 = dp[n - 1]
            ans2 = -1
            if dp[n - 2] == -1:
                ans2 = helper(n - 2, dp)
                dp[n - 2] = ans2
            else:
                ans2 = dp[n - 2]
            return ans1 + ans2
        dp = []
        for i in range(n):
            dp.append(-1)
        return helper(n, dp)
```

→ RECURSIVE

DP.

↓

→ T.C → $O(n)$.

call stack

↓

n+1 function

↓

Stack overflow

error

→ S.C → $O(n)$

↓

n + n

↓

dp.

call stack.

```
class Solution:
    def climbStairs(self, n: int) -> int:
        if n == 1 or n == 2:
            return n
```

→ T.C → $O(n)$

```
dp = []  
dp.append(1)  
dp.append(2)  
for i in range(2, n) :  
    dp.append(dp[i - 1] + dp[i - 2])  
return dp[n - 1]
```

→ S.C → $O(n)$
↓
dp array

↳ ITERATIVE DP